23L-0588 — VisionGuard

AI-generated versus real-image classification service

Student: Muhammad Abdul Raheem

Project: 23L-0588_A2_Ops

GitHub: github.com/abdul-raheem-fast/23L-0588_A2_Ops

DagsHub: dagshub.com/abdul-raheem-fast/23L-0588_A2_Ops

Live URL: 23l-0588-a2-ops.ghauri.prod.page

Final repository commit: e923d98

This report documents data understanding, model development, experiment tracking, Flask serving, database persistence, containerization, deployment, and real execution evidence for VisionGuard.

Disclosure

An AI assistant was used to help structure the implementation, identify reproducibility requirements, and review wording. All commands, runs, screenshots, account actions, metrics, and interpretations in this report must be produced and verified by the student.

1. Data understanding and versioning

1.1 Environment

The workflow was executed in an isolated Colab runtime and the environment was checked before training. Python, TensorFlow, Keras, Pillow, MLflow, and Docker versions were recorded in the run evidence.

The Colab runtime selected a T4 GPU. TensorFlow 2.18.0 was unavailable for that runtime, so the run used TensorFlow 2.21.0; this version change is recorded rather than hidden.

1.2 Dataset audit

The audit script records the class label, file size, dimensions, colour mode, file format, SHA-256 content hash, readability, and duplicate status for every file. This makes data-quality problems visible before the training split is created.

The Colab run audited 975 images: 539 from AiArtData and 436 from RealArt. No files were unreadable, and two exact duplicate files were identified by matching content hashes. The class imbalance is modest, so stratified splits were used to preserve class proportions.

1.3 Visual EDA

The audited metadata was used to generate dataset_eda.svg. The sample grid shows representative examples from both classes. The width, height, and file-size distributions show substantial variation, including very large source images, which supports resizing and batched loading in the input pipeline. These distributions describe the collected files rather than image semantics and are not treated as proof of a class-specific visual feature.

1.4 DVC

The repository includes the DVC pipeline configuration in dvc.yaml and params.yaml. The pipeline separates dataset auditing from split generation and records the seed and image size in configuration. A separate authenticated DVC push/pull screenshot was not captured, so remote transfer is not claimed as completed evidence.

2. Model development

The tf.data pipeline reads the reproducible split CSVs, decodes each image, converts it to RGB, resizes it to 224×224, batches samples, and prefetches batches for efficient execution. Training-only augmentation is applied before batching; validation and test images are only resized and normalized. Resizing uses a fixed square target, which provides a consistent tensor shape but can change the original aspect ratio.

The baseline CNN uses convolution, pooling, normalization, and dropout layers followed by a sigmoid output for binary classification. MobileNetV2 was then loaded without its classifier, trained first with the backbone frozen, and fine-tuned by unfreezing the final block at a lower learning rate. Early stopping, checkpointing on validation loss, and learning-rate reduction were used to retain the best generalizing checkpoint.

The executed Colab run selected the fine-tuned MobileNetV2 model. Its recorded test metrics are in champion_metrics.json: accuracy 0.7449, precision 0.8718, recall 0.6296, F1 0.7312, and ROC-AUC 0.8695. The precision/recall difference indicates that the selected threshold favours fewer false AI-generated predictions at the cost of missed positives. For this product, incorrectly calling a real photograph AI-generated is the more damaging user-facing error, so the precision-oriented operating point is preferable to maximizing recall alone. The evidence notebook generated confusion_matrix.png, roc_curve.png, error_analysis_grid.png, and error_analysis.csv.

3. MLflow and DagsHub

MLflow tracking was configured against DagsHub. Each executed experiment recorded the model family, training configuration, validation history, and final evaluation metrics, allowing the baseline and MobileNetV2 approaches to be compared from the run pages. The retained screenshots show the tracking configuration, completed runs, and final training output. Independent model-registry version and alias evidence was not captured.

4. Flask application

The model is loaded once during application startup. Upload validation checks the extension, size, and decoded image without persisting the upload. The result shows the label, confidence, and caveat; /history reads the request trail and /health reports service state.

5. Database trail

Each successful prediction stores a UTC timestamp, sanitized filename, decision, confidence, model name, and model version. Uploaded image contents are not persisted. SQLite is used by the free deployment, while the application also supports PostgreSQL through environment-based configuration.

6. Docker and Compose

The Docker image installs serving dependencies separately from training and MLflow tools. It uses a slim CPU base, runs as a non-root user, downloads the public champion model when it is absent from the build context, and exposes port 8000. Compose defines the web and database services, health checks, readiness dependency, and named volumes. The local Compose run reached healthy status for both services, and the web health endpoint returned the expected model and version response. The production image includes the PostgreSQL driver required by the Compose database configuration.

The first local build exposed two compatibility issues: the installed Gunicorn version did not support the --factory flag, and the database layer used SQLite parameter placeholders with PostgreSQL. The command was changed to Gunicorn's callable-factory syntax, the PostgreSQL driver was added to the serving requirements, and database schema and query placeholders were made backend-aware. The

corrected stack then started successfully.

7. API deployment

The application was deployed from the GitHub main branch on Blitz Cloud using the repository Dockerfile. The deployment uses the public address https://23l-0588-a2-ops.ghauri.prod.page. The build initially failed because the training manifest combined TensorFlow 2.21.0 with MLflow 2.19.0, which require incompatible protobuf ranges. The production and training dependencies were separated, the updated image built successfully, and the public /health endpoint returned the deployed model name, version, and ok status. The free service may sleep when idle; no paid cloud credits were used.

8. Flask and hosted API integration

The deployed Flask service accepts image uploads at /predict, renders the prediction result, and records successful requests for /history. A real dataset image was sent to the public endpoint and returned an AI-generated decision with 71.3% confidence. The health endpoint was checked independently from outside the local workspace and returned the deployed model name, version, and an ok status. The service uses SQLite for the free deployment because no managed database was provisioned; production deployments should use PostgreSQL and a secret-managed configuration.

Submission checklist

• [x] PDF renamed to 23L-0588_VisionGuard.pdf

• [x] GitHub repository link

• [x] DagsHub project link

• [x] Live API URL tested from another network

• [x] Every retained evidence item is a real, captioned screenshot

• [x] No credentials appear in source, screenshots, or report

Appendix A — Saved evidence screenshots

• 01_colab_runtime_t4.png — Colab runtime configuration with a T4 GPU selected.

• 02_colab_secrets_panel.png — Colab Secrets setup panel for DagsHub credentials.

• 10_github_ci_failed.png — Intentional red GitHub Actions run retained for CI lab evidence.

• 15_colab_environment_gpu.png — Colab environment output showing TensorFlow and GPU detection.

• 16_dataset_download_counts.png — Official dataset download path and class file counts.

• 17_dataset_audit_splits.png — Audit count and reproducible train/validation/test split creation.

• 18_dagshub_mlflow_configuration.png — DagsHub MLflow tracking configuration with the token hidden.

• 19_baseline_training_result.png — Baseline CNN training output and evaluation metrics.

• 20_dagshub_final_runs.png — DagsHub MLflow experiment list showing completed VisionGuard runs.

• 21_finetuned_training_result.png — MobileNetV2 fine-tuning output and evaluation metrics.

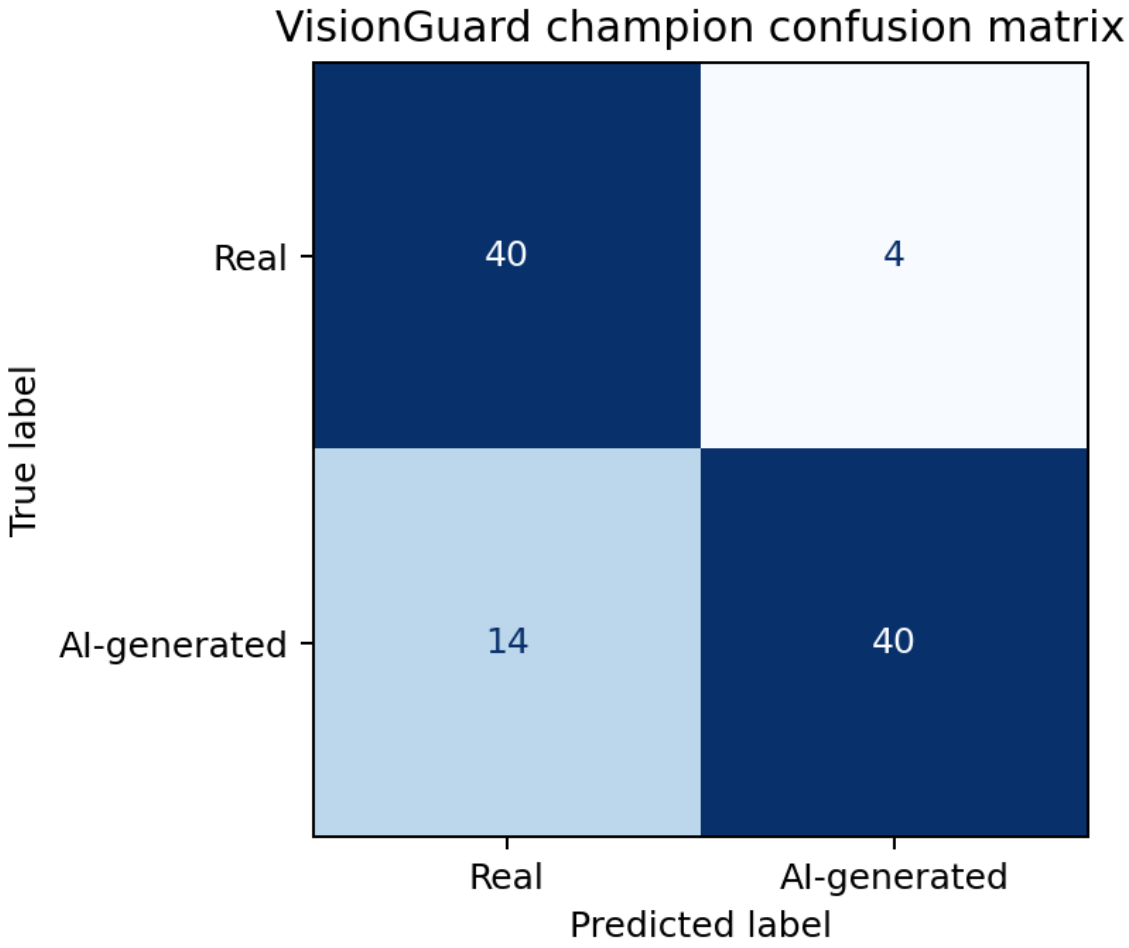
• 22_huggingface_space_files.png — Hugging Face Space file listing captured while diagnosing deployment layout.

• 23_google_cloud_trial_pending.png — Google Cloud free trial pending because prepayment verification was required.

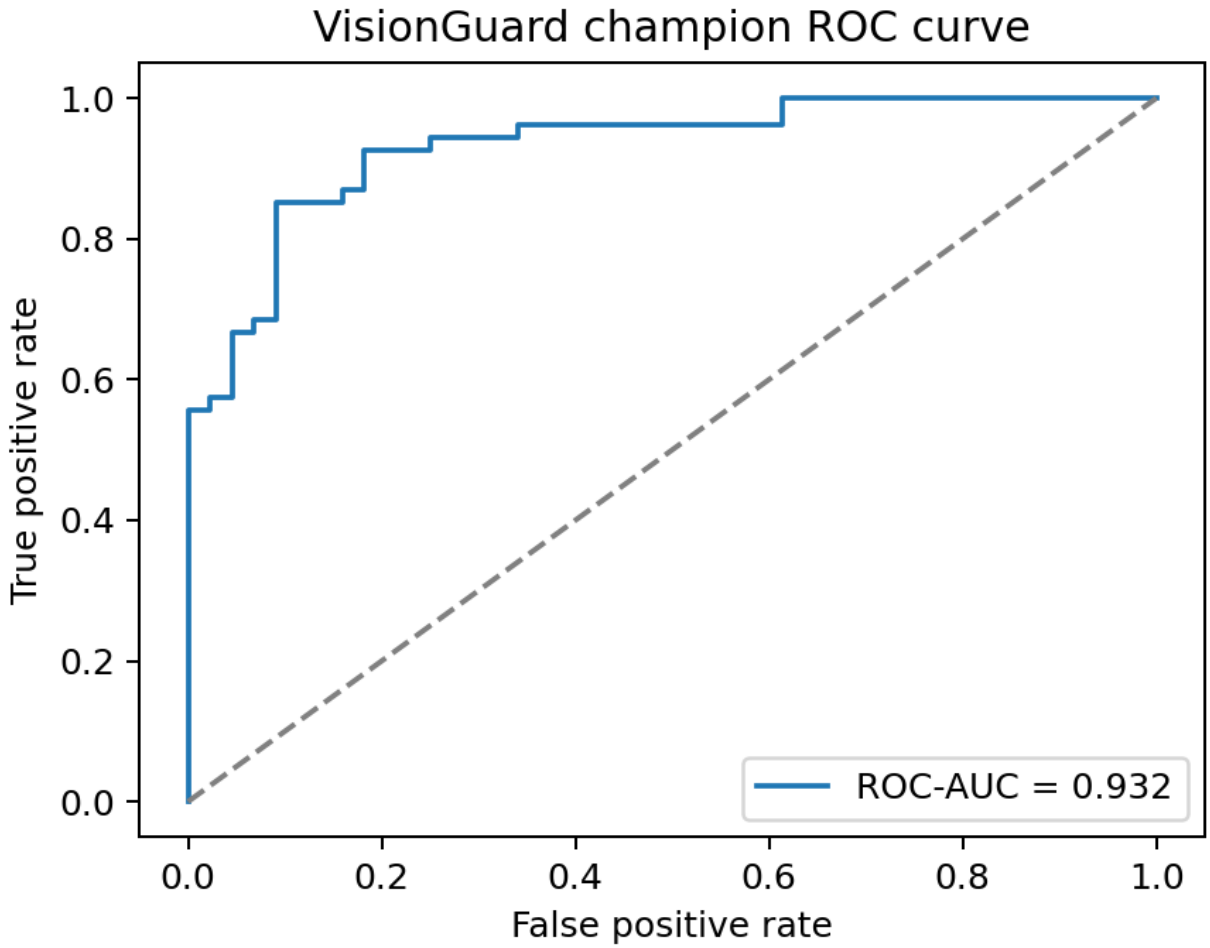
• 24_blitz_dashboard_online.png — Blitz Cloud dashboard showing the public deployment online.

• 25_blitz_public_ui.png — Public Blitz URL displaying the VisionGuard upload interface.

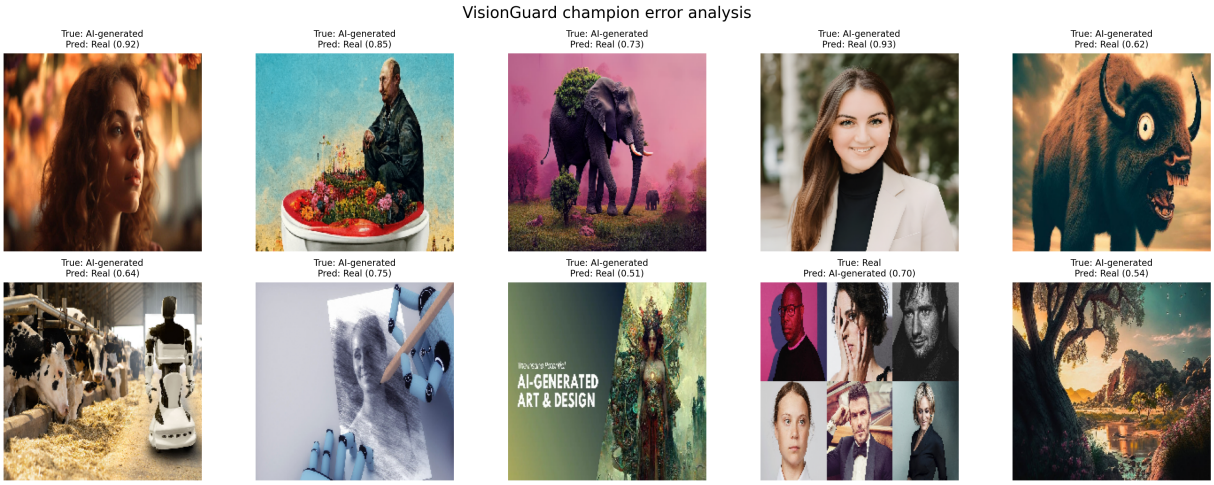
Evaluation figures



confusion_matrix.png *— Champion confusion matrix from the Colab evidence run.*

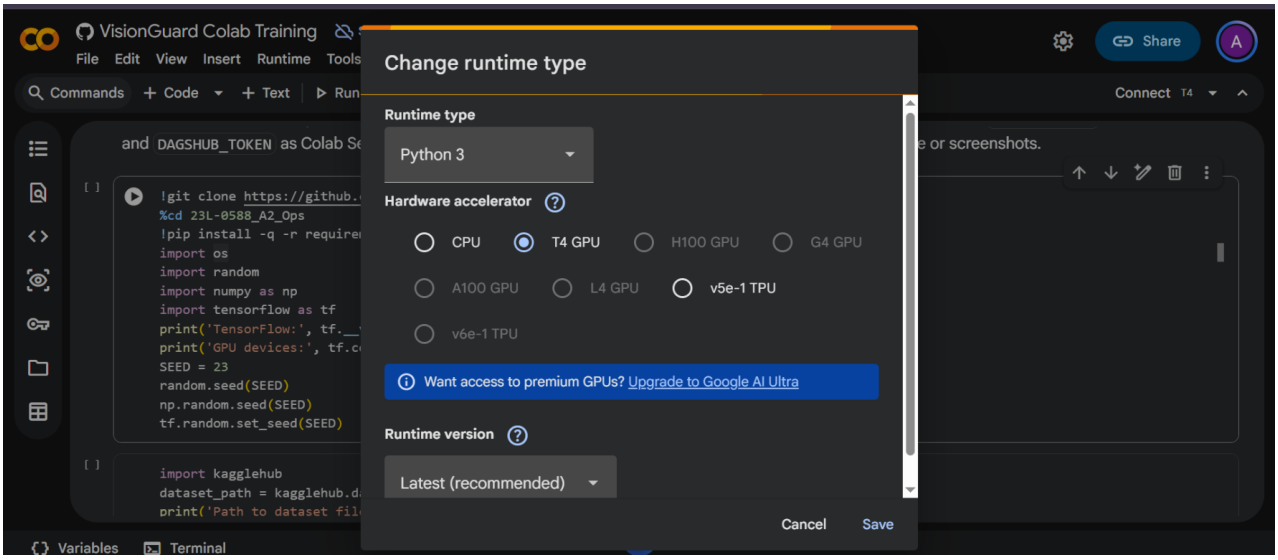


roc_curve.png *— Champion ROC curve and area-under-curve result.*

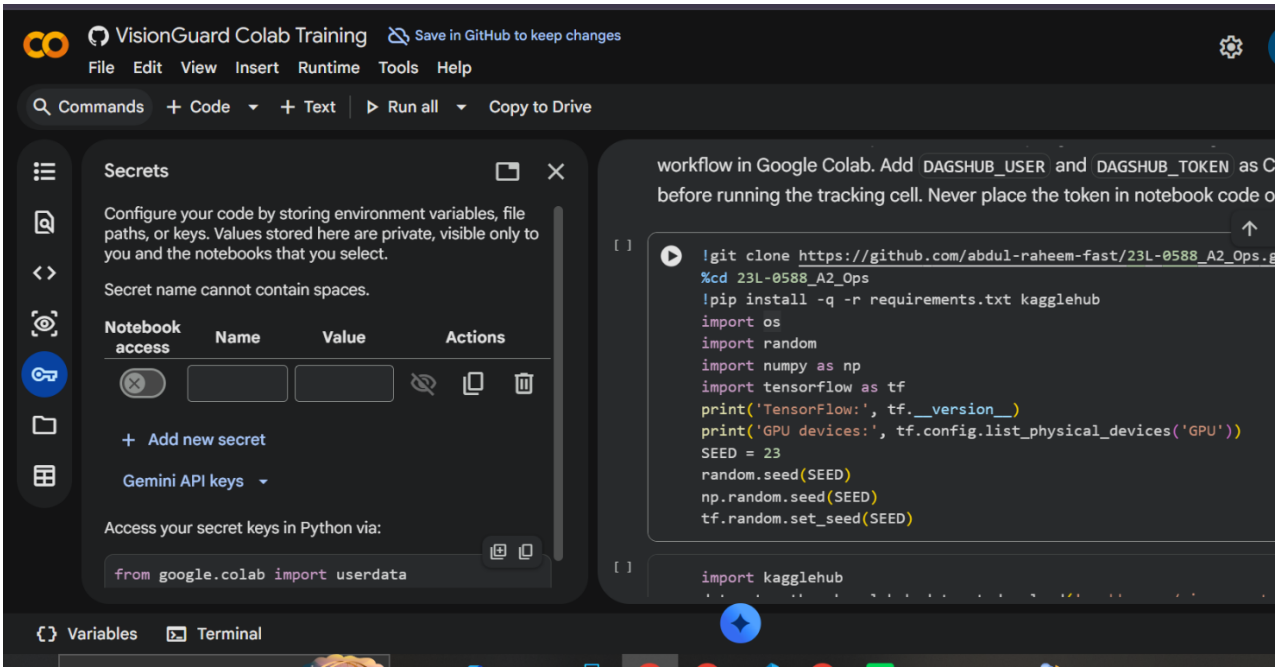


error_analysis_grid.png *— Ten selected test-set errors used for qualitative analysis.*

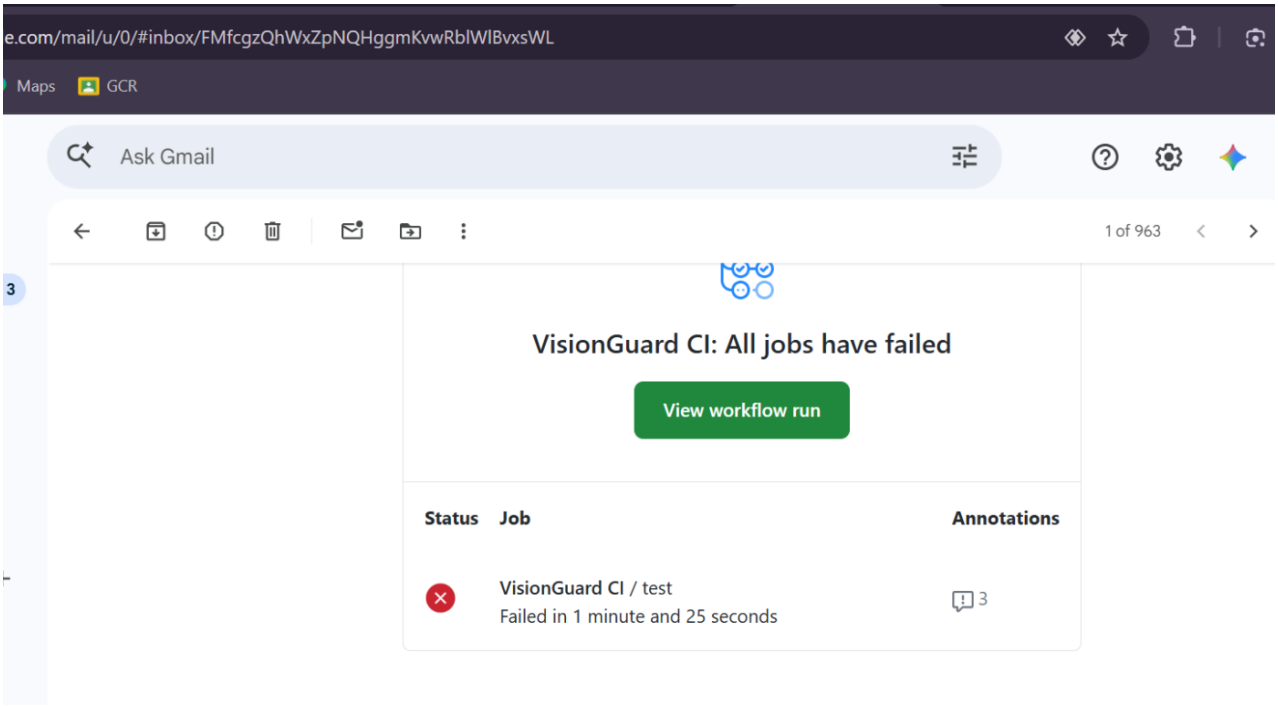
Appendix — Saved evidence screenshots



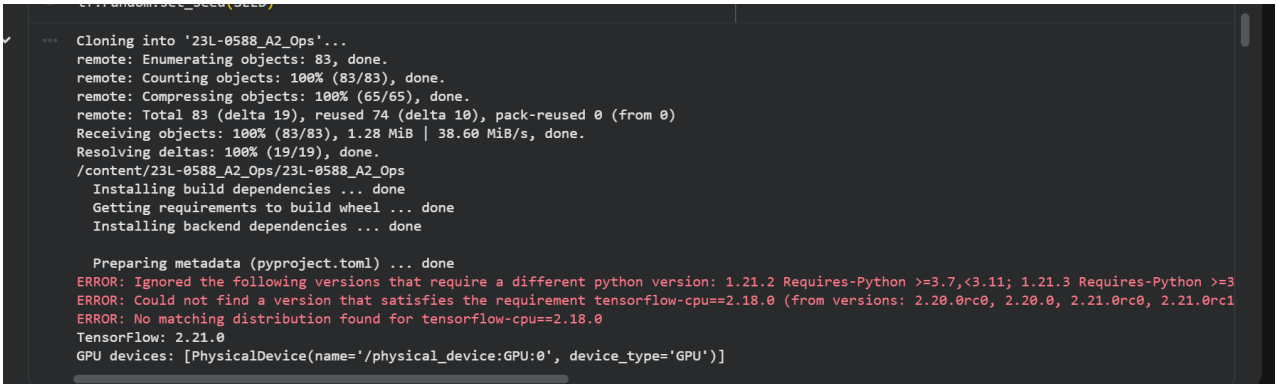
01_colab_runtime_t4.png *— Colab runtime configuration with a T4 GPU selected.*



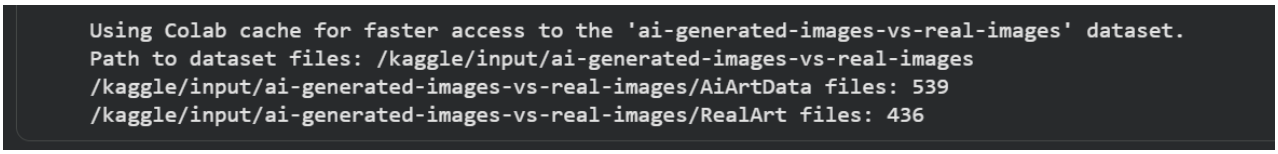
02_colab_secrets_panel.png *— Colab Secrets setup panel for DagsHub credentials.*



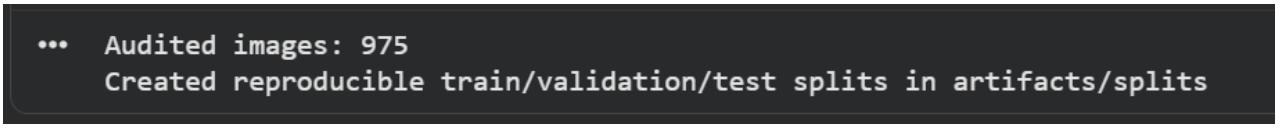
10_github_ci_failed.png *— Intentional red GitHub Actions run retained for CI lab evidence.*



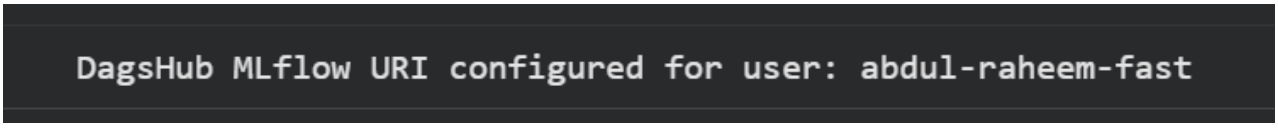
15_colab_environment_gpu.png *— Colab environment output showing TensorFlow and GPU detection.*



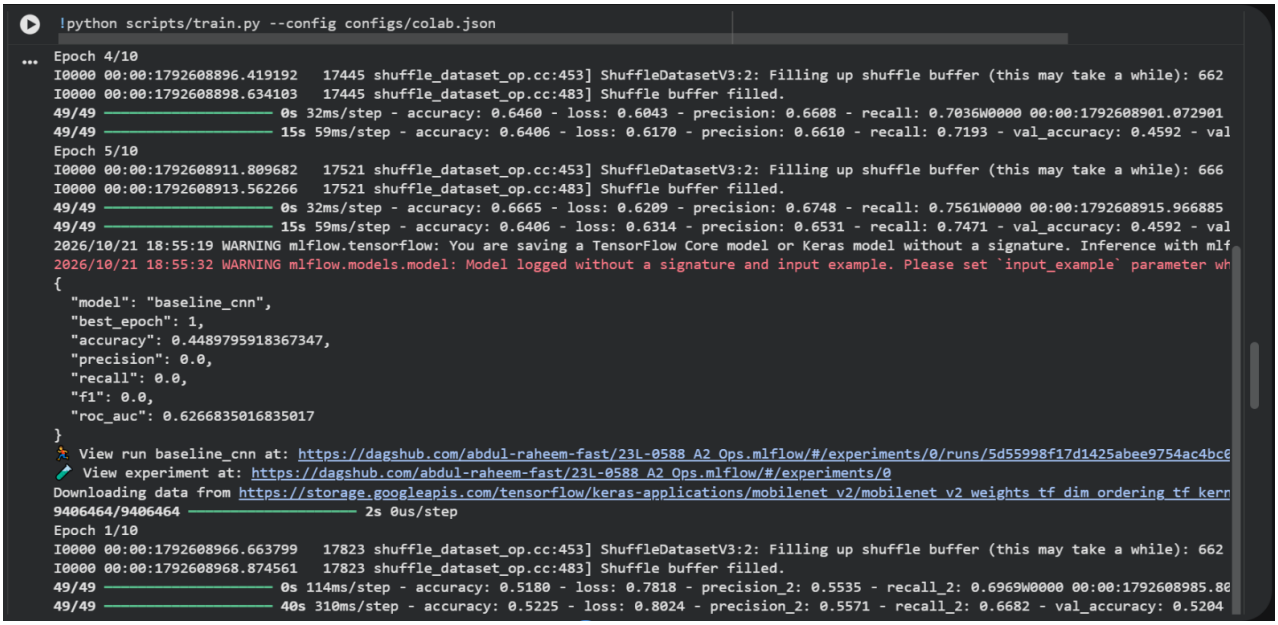
16_dataset_download_counts.png *— Official dataset download path and class file counts.*



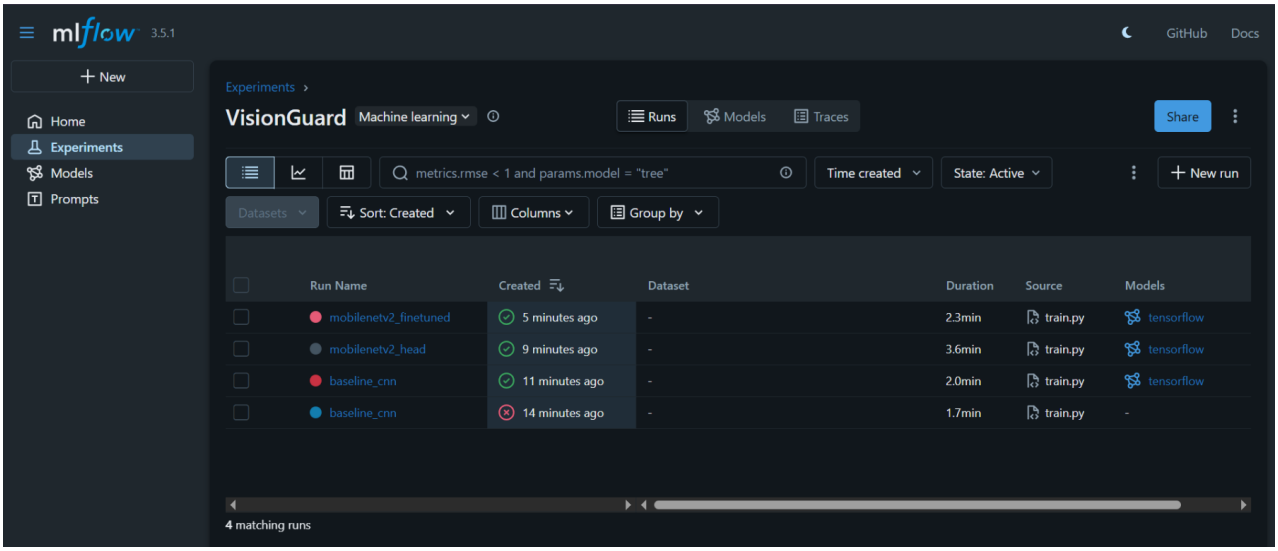
17_dataset_audit_splits.png *— Audit count and reproducible train/validation/test split creation.*



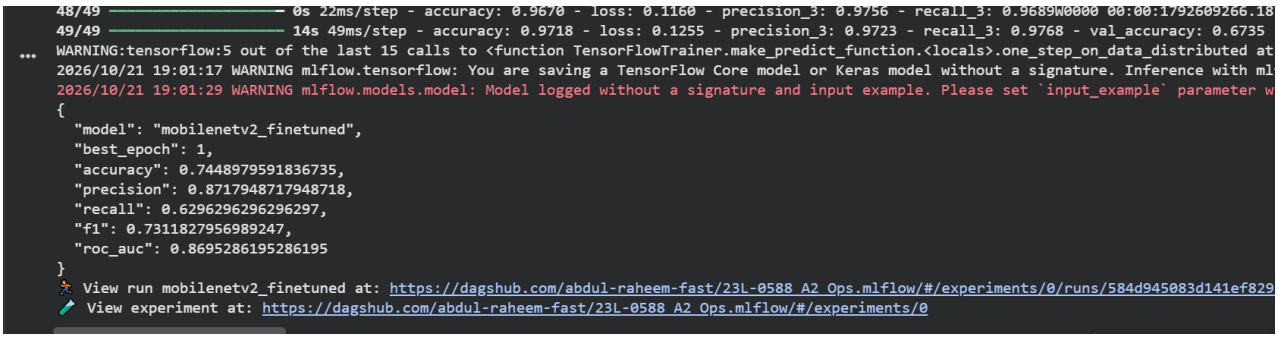
18_dagshub_mlflow_configuration.png *— DagsHub MLflow tracking configuration with the token hidden.*



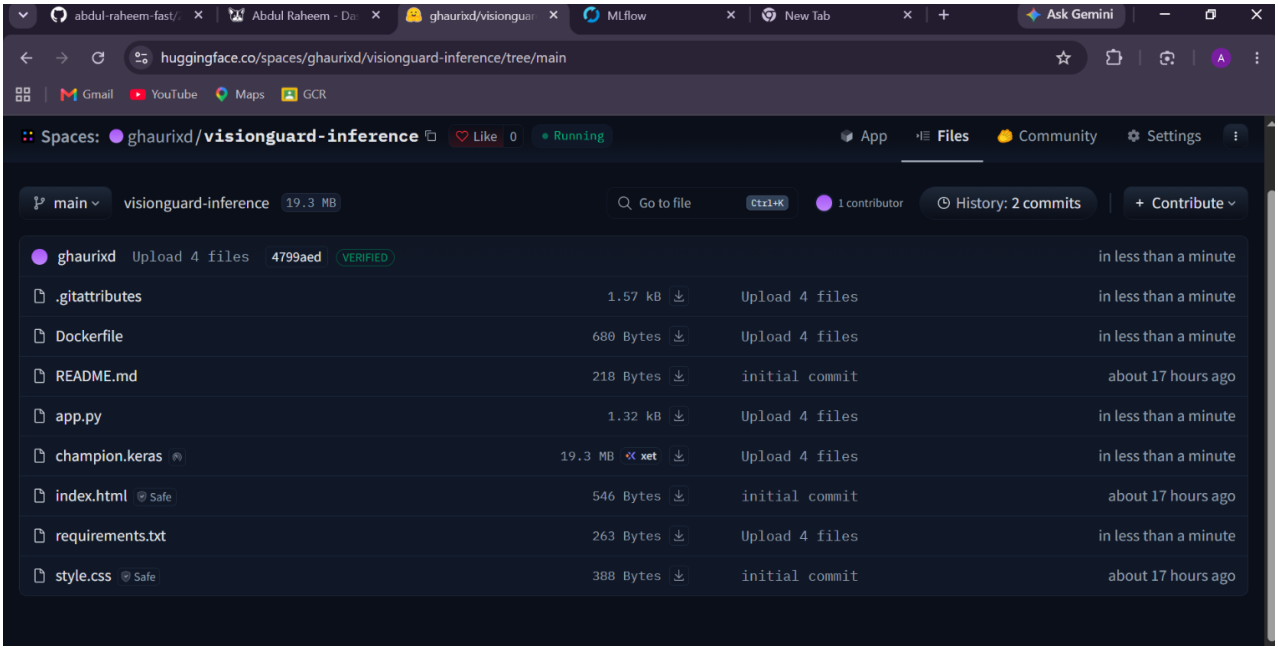
19_baseline_training_result.png *— Baseline CNN training output and evaluation metrics.*



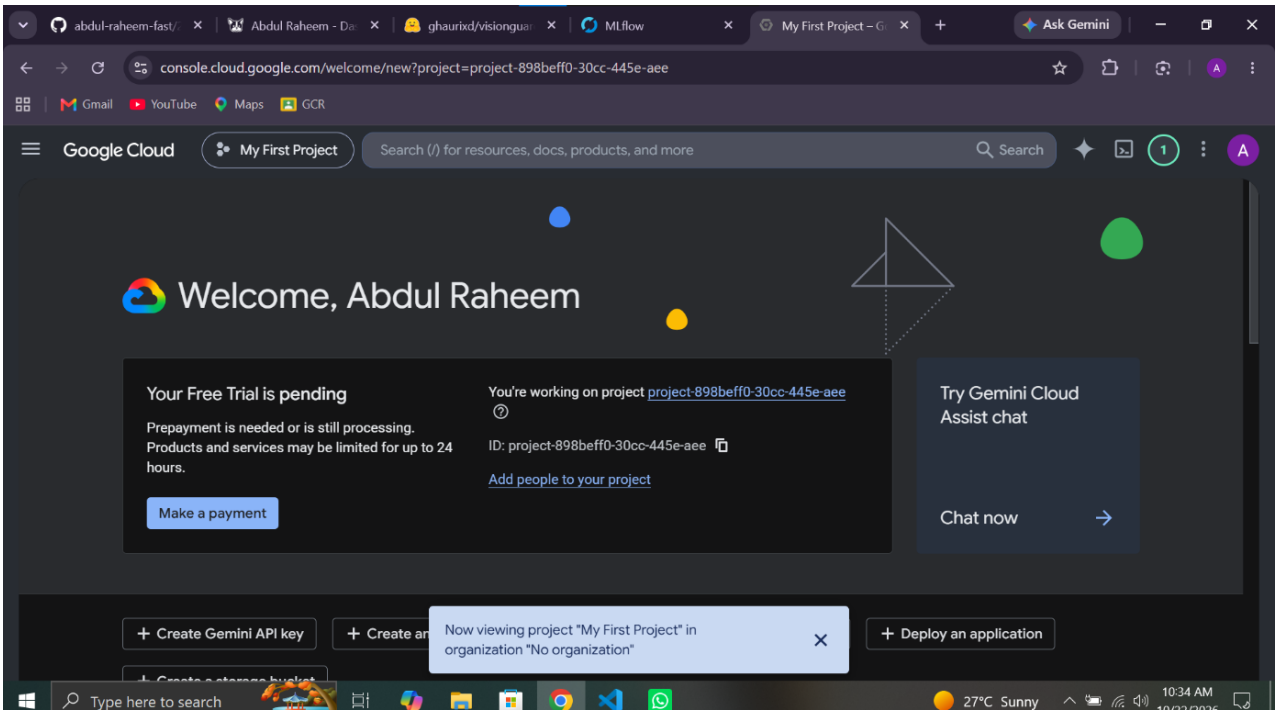
20_dagshub_final_runs.png *— DagsHub MLflow experiment list showing completed VisionGuard runs.*



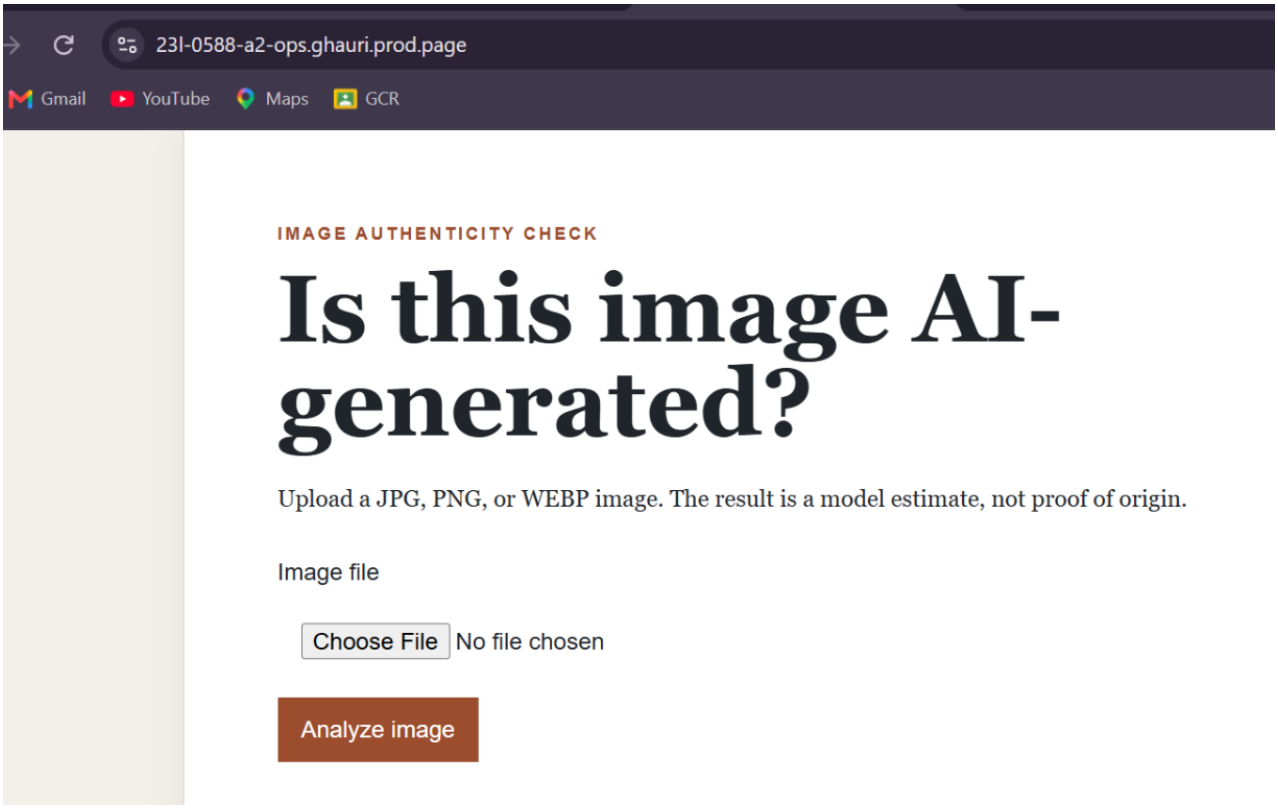
21_finetuned_training_result.png *— MobileNetV2 fine-tuning output and evaluation metrics.*



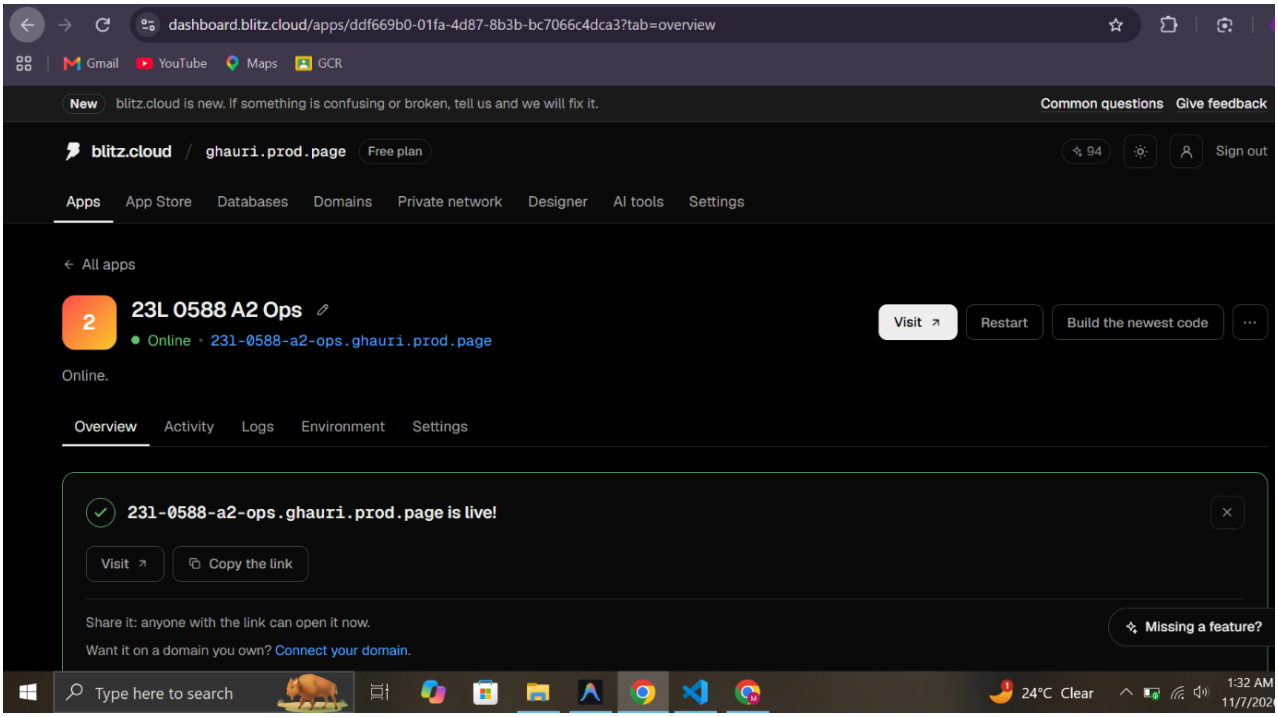
22_huggingface_space_files.png *— Hugging Face Space file listing captured while diagnosing the deployment layout.*



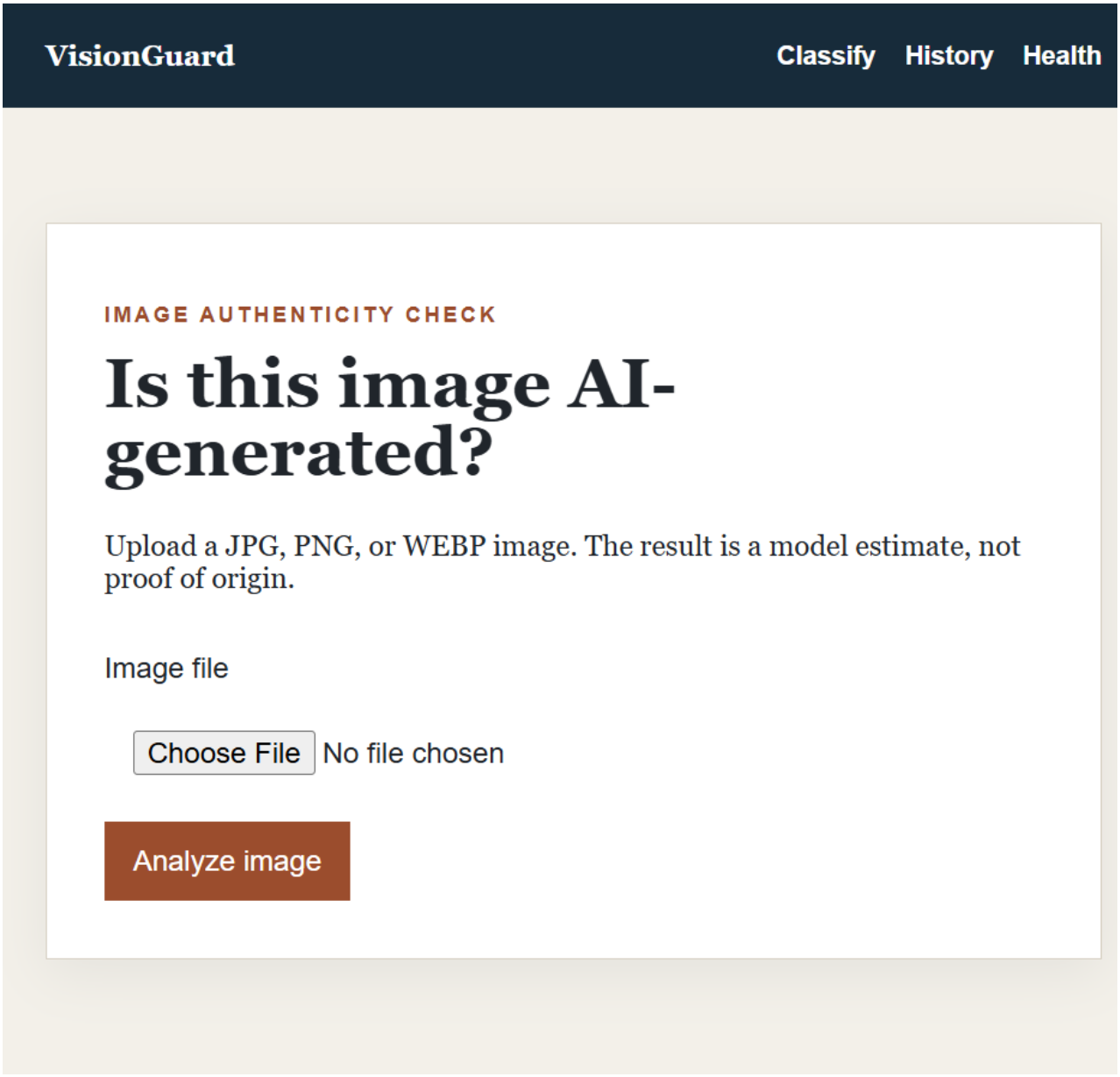
23_google_cloud_trial_pending.png *— Google Cloud free trial pending because prepayment verification was required.*



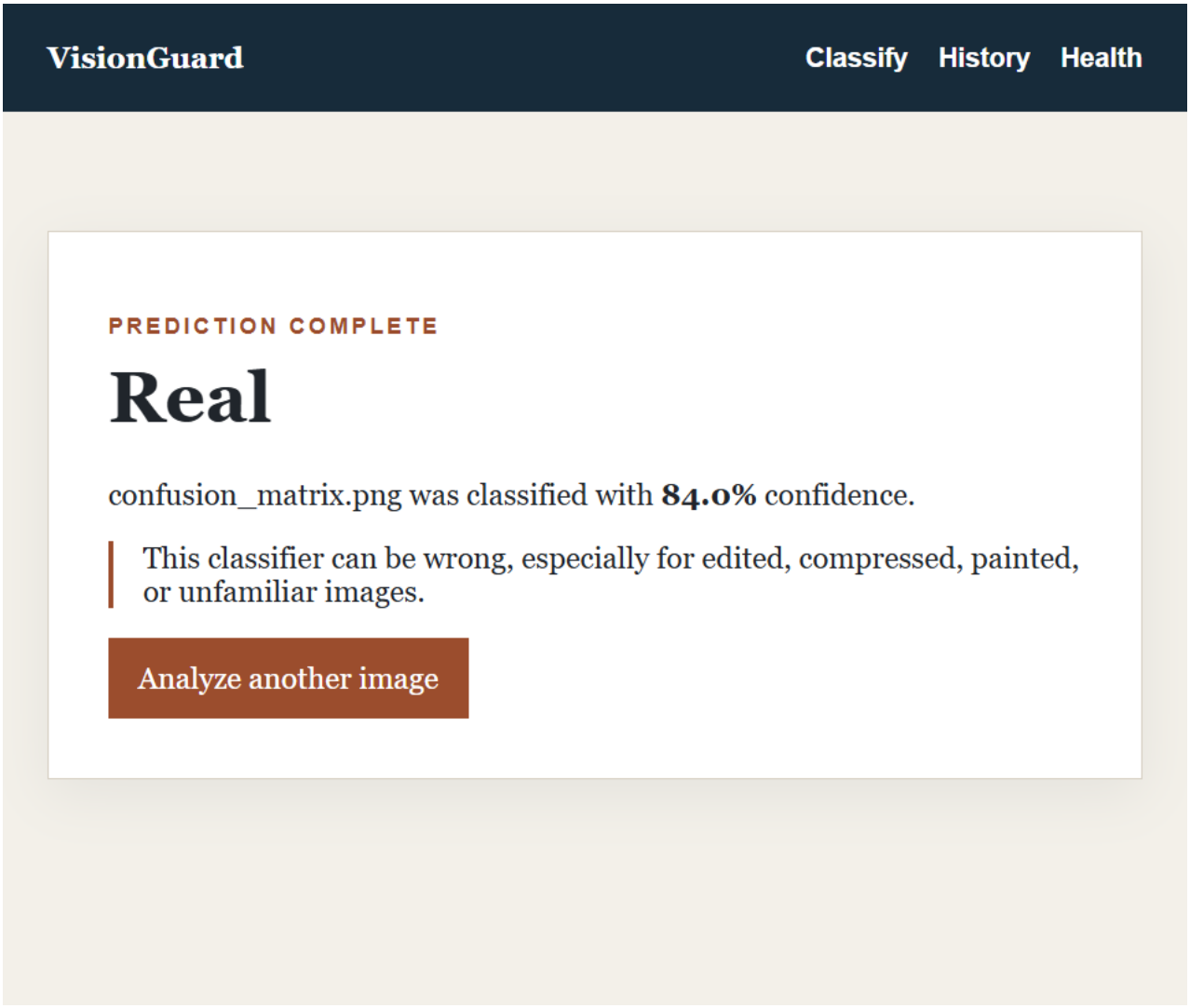
24_blitz_dashboard_online.png *— Blitz Cloud dashboard showing the deployed application online with its public address.*



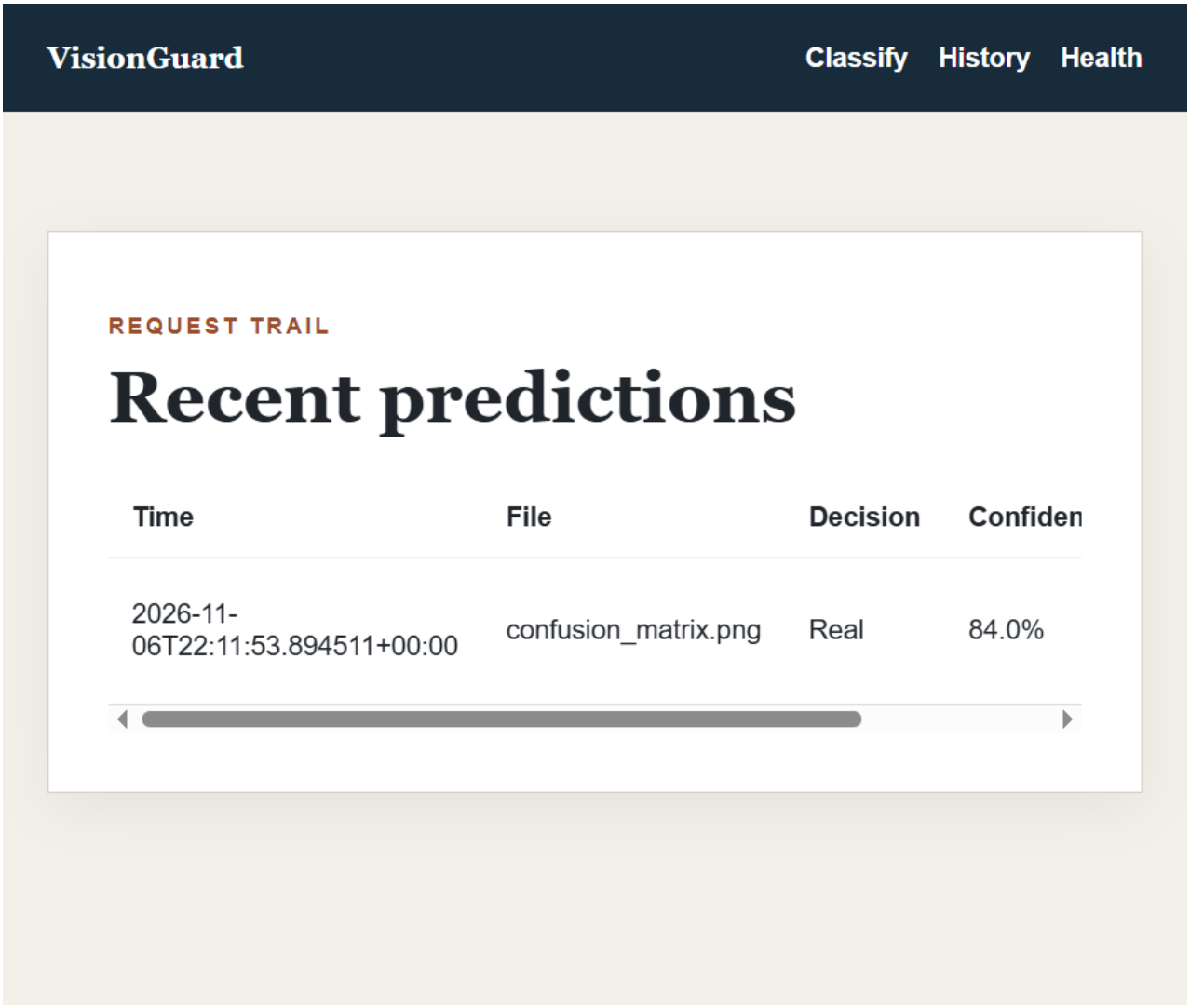
25_blitz_public_ui.png *— Public Blitz URL displaying the VisionGuard upload interface.*



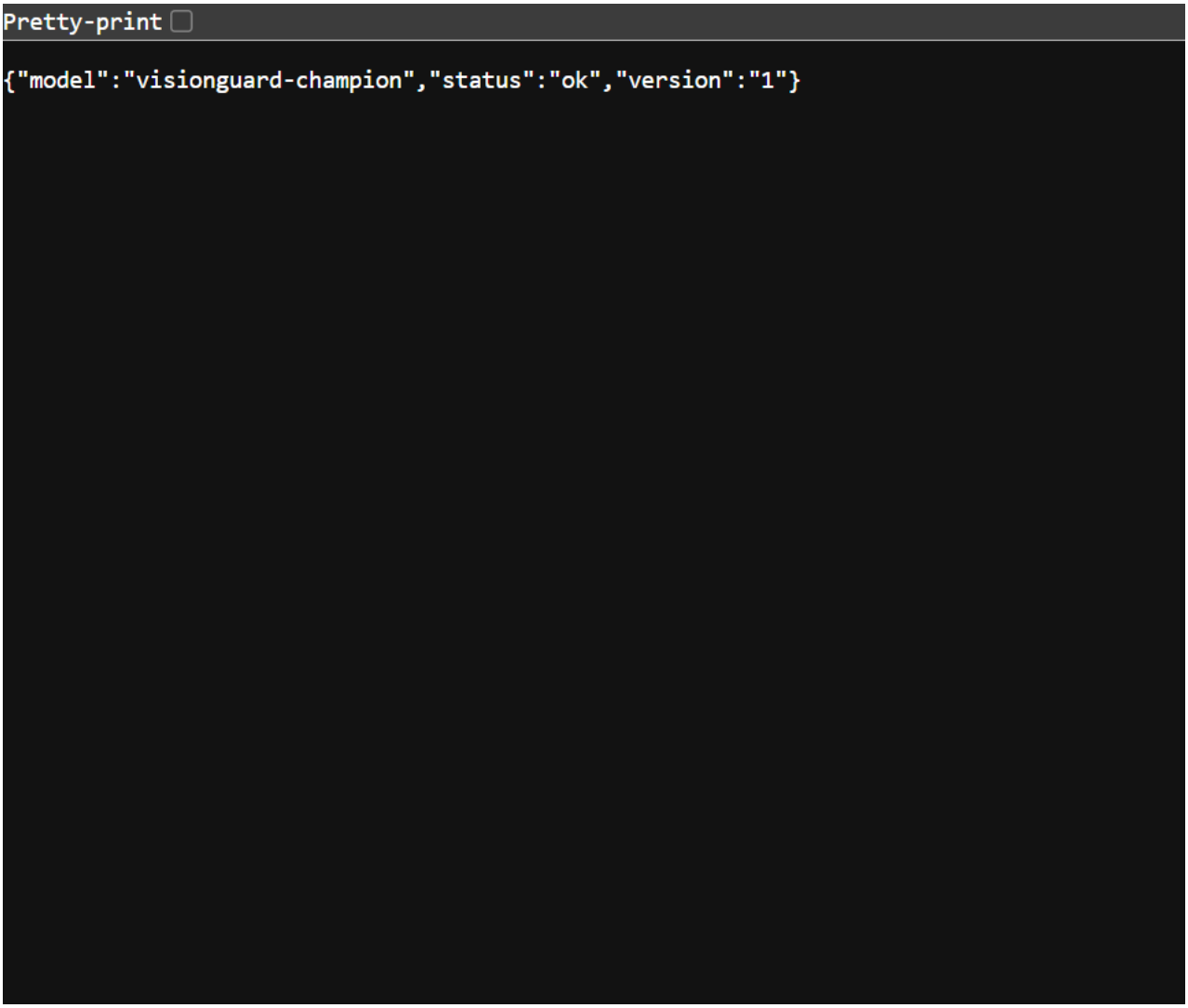
26_compose_local_ui.png *— Tasks 4.2, 4.3, and 6.3 — local Docker Compose interface with the upload, history, and health routes.*



27_compose_prediction_result.png *— Tasks 4.1, 4.2, and 8.1 — real local prediction result showing the label, confidence, and model caveat.*



28_compose_history.png *— Tasks 5 and 8.1 — database-backed history showing timestamp, filename, decision, confidence, model, and version.*



29_compose_health.png *— Tasks 4.4 and 6.3 — health response from the Compose web service showing model, version, and status.*